

Лабораторная работа №3:

Условная и негладкая оптимизация.

1 Введение

В предыдущих лабораторных работах мы рассматривали задачи безусловной минимизации гладких функций потерь $f(x)$. Однако в машинном обучении мы часто хотим, чтобы модель обладала свойством разреженности – то есть, чтобы большинство весов x_i были строго равны нулю. Это позволяет проводить отбор признаков, ускоряет инференс и делает модель интерпретируемой.

Для индуцирования разреженности к гладкой функции потерь добавляют негладкий регуляризатор, чаще всего L_1 -норму. Таким образом, общая задача принимает вид:

$$\min_{x \in \mathbb{R}^n} F(x) := f(x) + \lambda \|x\|_1 \quad (1)$$

где $f(x)$ — гладкая выпуклая функция потерь (ваш ML-оракул), а $\lambda > 0$ — коэффициент регуляризации.

Уникальность задачи (1) заключается в том, что её можно решать тремя принципиально разными подходами:

- методами негладкой оптимизации;
- методами условной оптимизации (переформулировав штраф как ограничение);
- методами внутренней точки (сгладив ограничения барьером).

2 Алгоритмы

2.1 Негладкая оптимизация

2.1.1 Композитная оптимизация

Задача, описанная во введении относится к классу композитной оптимизации, которая представляет собой задачу минимизации специального вида:

$$\min_{x \in \mathbb{R}^n} f(x) + h(x)$$

где $f : \mathbb{R}^n \rightarrow \mathbb{R}$ - дифференцируемая функция с липшицевым градиентом, $h : \mathbb{R}^n \rightarrow \mathbb{R}$ - простая выпуклая замкнутая функция.

Под простотой функции h подразумевается то, что возможно эффективно вычислить проксимальный оператор

$$\text{prox}_{\lambda h}(x) := \underset{u}{\operatorname{argmin}} \left\{ \lambda h(u) + \frac{1}{2} \|u - x\|^2 \right\}$$

для любого $\lambda > 0$ и любого $x \in \mathbb{R}^n$. Например, для функции $\|\cdot\|_1 : \mathbb{R}^n \rightarrow \mathbb{R}$ задача вычисления проксимального оператора распадается на n независимых одномерных задач

минимизации, которые имеют точное аналитическое решение. Этот оператор называется Soft-Thresholding (оператор мягкого порога):

$$[\text{prox}_{\lambda \|\cdot\|_1}(x)]_i = \begin{cases} x_i - \lambda, & x_i > \lambda \\ 0, & -\lambda \leq x_i \leq \lambda \\ x_i + \lambda, & x_i < -\lambda \end{cases} = \text{sign}(x_i) \max(|x_i| - \lambda, 0) \quad (2)$$

Именно наличие операции $\max(\dots, 0)$ гарантирует, что малые веса строго обнуляются, обеспечивая истинную разреженность. (2) можно переписать в другом виде:

$$\text{prox}_{\lambda \|\cdot\|_1}(x) = (\text{sign}(x_i) [|x_i| - \lambda]_+)_{1 \leq i \leq n}$$

для $x \in \mathbb{R}^n, \lambda > 0$, где $[\cdot]_+ : \mathbb{R} \rightarrow \mathbb{R}$ — положительная срезка $[t]_+ := \max\{t, 0\}$.

2.1.2 Субградиентный метод

Самый наивный подход к задаче (??) — воспринимать $F(x)$ как единую негладкую функцию и использовать обобщение градиента.

Субградиентом выпуклой функции $F(x)$ в точке x_0 называется любой вектор g , удовлетворяющий неравенству $F(x) \geq F(x_0) + \langle g, x - x_0 \rangle$ для всех x .

Множество всех субградиентов называется **субдифференциалом** $\partial F(x_0)$.

Для L_1 -регуляризатора субдифференциал вычисляется покомпонентно:

$$[\partial \|x\|_1]_i = \begin{cases} 1, & x_i > 0 \\ -1, & x_i < 0 \\ [-1, 1], & x_i = 0 \end{cases} \quad (3)$$

Итерация субградиентного метода заключается в шаге из текущей точки x_k в направлении (произвольного) анти-субградиента:

$$x_{k+1} = x_k - \alpha_k g_k, \quad \text{где } g_k \in \partial \|x\|_1(x_k) \quad (4)$$

Проблема метода: Из-за негладкости метод не может сходиться с постоянным шагом. Для сходимости метода необходимо, чтобы длины шагов α_k убывали к нулю, но не слишком быстро:

$$\alpha_k > 0, \quad \alpha_k \rightarrow 0, \quad \sum_{k=0}^{\infty} \alpha_k = \infty$$

Обычно длины шагов выбирают по правилу $\alpha_k = \alpha / \sqrt{k+1}$, где $\alpha > 0$ — некоторая константа.

Нужно отметить, что последовательность $(x_k)_{k=0}^{\infty}$, построенная субградиентным методом, может не быть (и, как правило, зачастую не является) релаксационной последовательностью для функции F , т. е. неравенство $F(x_{k+1}) \leq F(x_k)$ может не выполняться. Поэтому в субградиентном методе в качестве результата работы после N итераций метода вместо точки x_N возвращается точка $y_N := \arg\min\{F(x) : x \in \{x_0, \dots, x_N\}\}$ (т. е. из всех пробных точек x_k , построенных методом, выбирается та, в которой значение функции оказалось наименьшим¹

Стоит отметить, что из-за структуры субградиента метод никогда не достигает точного нуля по координатам: перескакивая через ноль, знак субградиента меняется на противоположный, вызывая бесконечные осцилляции (зигзаги) вокруг нуля. Таким образом, метод не способен выдать истинно разреженное решение.

¹Заметим, что в практической реализации метода для вычисления результата y_N сами точки $x_0, \dots, x_N$ хранить в памяти не нужно.

2.1.3 Проксимальный Градиентный метод

Проксимальный градиентный метод – это адаптация градиентного спуска для композитных задач. Частный случай метода, когда $h(x) = \lambda\|x\|_1$, называется ISTA (Iterative Shrinkage-Thresholding Algorithm).

Идея состоит в том, чтобы на каждом шаге аппроксимировать гладкую часть $f(x)$ параболой (как в градиентном спуске), а негладкую $h(x)$ оставить без изменений.

Минимизация такой мажоранты математически сводится к применению проксимального оператора к точке, полученной после обычного шага градиентного спуска:

$$x_{k+1} = \text{prox}_{\alpha_k \lambda \|\cdot\|_1} \left(\underbrace{x_k - \alpha_k \nabla f(x_k)}_{\text{шаг градиентного спуска}} \right) \quad (5)$$

Здесь α_k – длина шага. Если константа Липшица L градиента $\nabla f(x)$ известна, то $\alpha_k \leq \frac{1}{L}$. Если нет – используется бэктрекинг по условию убывания функции. Скорость сходимости ISTA для выпуклых функций составляет $\mathcal{O}(1/k)$, как и у обычного градиентного спуска.

Algorithm 1 ISTA

- 1: **Инициализация:** $x_0 \in \mathbb{R}^n$, L_0
 - 2: **for** $k = 0, 1, 2, \dots$ **do**
 - 3: $\bar{L}_k := L_k$
 - 4: $x_{k+1} = \text{prox}_{\lambda \|\cdot\|_1 / \bar{L}_k} \left(x_k - \frac{1}{\bar{L}_k} \nabla f(x_k) \right)$
 - 5: Если $f(x_{k+1}) > f(x_k) + \langle \nabla f(x_k), x_{k+1} - x_k \rangle + \frac{\bar{L}_k}{2} \|x_{k+1} - x_k\|^2$, положить $\bar{L}_k := 2\bar{L}_k$ и вернуться к шагу 4.
 - 6: Положить $L_{k+1} := \bar{L}_k / 2$
 - 7: **end for**
-

Несмотря на то, что на отдельных шагах этой схемы может выполняться достаточно много итераций одномерного поиска по подбору параметра $\bar{L}_k$, можно показать, что среднее число итераций одномерного поиска за итерацию метода примерно равно двум. Параметр L_0 , по сути дела, влияет лишь на число одномерных поисков на самых первых итерациях метода; его всегда можно выбрать равным единице ($L_0 = 1$) без принципиального ущерба для скорости сходимости метода, поскольку в итоговую оценку суммарной трудоемкости метода этот параметр входит под логарифмом.

2.1.4 Быстрый проксимальный градиентный метод

Метод FISTA (Fast Iterative Shrinkage-Thresholding Algorithm), предложенный Беком и Тебуллем в 2009 году, является ускоренной версией ISTA. Он применяет идею инерции Нестерова к проксимальному градиентному спуску, что позволяет улучшить скорость сходимости для выпуклых задач с $\mathcal{O}(1/k)$ до теоретически оптимальной $\mathcal{O}(1/k^2)$.

В классической нотации Нестерова метод оперирует тремя последовательностями точек:

- x_k — основная последовательность приближений.
- v_k — последовательность инерции (минимум модельной оценивающей функции).
- y_k — точка экстраполяции, в которой вычисляется градиент.

Также используются весовые коэффициенты A_k и a_k , которые регулируют «шаг времени» метода. Алгоритм выглядит следующим образом:

Вместо того чтобы делать шаг из предыдущей точки x_k , метод вычисляет градиент в экстраполированной точке y_k , которая учитывает "скорость" движения с предыдущих шагов. Алгоритм выглядит следующим образом:

Algorithm 2 Быстрый композитный градиентный метод Нестерова (FISTA)

- 1: **Вход:** Начальная точка $x_0 \in \mathbb{R}^n$.
- 2: **Инициализация:** $v_0 = x_0$, $A_0 = 0$, $L_0 > 0$.
- 3: **for** $k = 0, 1, \dots$ **do**
- 4: $\bar{L}_k := L_k$
- 5: Найти корень $a_{k+1} > 0$ из квадратного уравнения: $\frac{a_{k+1}^2}{A_k + a_{k+1}} = \frac{1}{\bar{L}_k}$.
 (Явное решение: $a_{k+1} = \frac{1 + \sqrt{1 + 4\bar{L}_k A_k}}{2\bar{L}_k}$)
- 6: Обновить полный вес: $A_{k+1} = A_k + a_{k+1}$
- 7: Вычислить точку экстраполяции (выпуклая комбинация):

$$y_k = \frac{A_k x_k + a_{k+1} v_k}{A_{k+1}}$$

- 8: Сделать градиентный шаг с проксимальным оператором:

$$x_{k+1} = \text{prox}_{\lambda \|\cdot\|_1 / \bar{L}_k} \left(y_k - \frac{1}{\bar{L}_k} \nabla f(y_k) \right)$$

- 9: Если $f(x_{k+1}) > f(y_k) + \langle \nabla f(y_k), x_{k+1} - y_k \rangle + \frac{\bar{L}_k}{2} \|x_{k+1} - y_k\|^2$, положить $\bar{L}_k := 2\bar{L}_k$ и вернуться к шагу 5.
- 10: Положить $L_{k+1} := \bar{L}_k / 2$
- 11: Обновить последовательность инерции:

$$v_{k+1} = v_k + \frac{A_{k+1}}{a_{k+1}} (x_{k+1} - y_k)$$

- 12: **end for**
-

Физический смысл шага 7: точка y_k смещается в сторону инерции v_k , и именно в этой предсказанной точке мы прощупываем функцию градиентом.

Смысл шага 11: если шаг алгоритма $(x_{k+1} - y_k)$ оказался удачным, мы усиливаем вектор инерции v_{k+1} в этом направлении с коэффициентом $\frac{A_{k+1}}{a_{k+1}}$, который растет с каждой итерацией. Именно этот механизм накопления дает взрывное ускорение метода.

В отличие от обычного градиентного метода, быстрый градиентный метод работает уже с несколькими последовательностями точек. При этом вдоль каждой из этих последовательностей целевая функция может уменьшаться не монотонно. Поэтому в качестве ответа $\bar{x}_k$ метода следует выдавать ту точку, в которой наблюдалось наименьшее (так называемое рекордное) значение целевой функции среди всех точек x_k, y_k , в которых выполнялись вычисления функции.

2.2 Условная оптимизация

2.2.1 Переформулировка задачи

В этом разделе мы рассматриваем альтернативный взгляд на проблему поиска разреженных весов. Вместо того чтобы добавлять негладкий штраф в целевую функцию, мы можем переформулировать задачу как минимизацию гладкой функции на замкнутом выпуклом множестве (в нашем случае – на L_1 -шаре):

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{при условии} \quad \|x\|_1 \leq R \quad (6)$$

где $R > 0$ — радиус шара, который является гиперпараметром (существует взаимно однозначное соответствие между параметром λ из безусловной задачи и радиусом R).

Геометрически L_1 -шар представляет собой гипероктаэдр, вершины которого лежат строго на координатных осях. Линии уровня выпуклой функции потерь $f(x)$ с высокой вероятностью коснутся этого многогранника не в его внутренней точке, а на грани низкой размерности (или прямо в вершине). Это свойство геометрии L_1 -шара и гарантирует получение строгих нулей в векторе x .

2.2.2 Метод Франк-Вульфа (Условного градиента)

Классическим методом решения условных задач является метод проекций градиента. Однако проекция на L_1 -шар вычислительно сложна (требует сортировки компонент за $\mathcal{O}(n \log n)$).

Метод Франк-Вульфа (Frank-Wolfe, или метод Условного градиента) позволяет полностью избежать операции проекции. Основная идея метода заключается в том, чтобы на каждой итерации заменить целевую функцию $f(x)$ её линейной аппроксимацией Тейлора первого порядка в текущей точке x_k :

$$f_{x_k}^I(s) \approx f(x_k) + \langle \nabla f(x_k), s - x_k \rangle$$

Затем метод ищет точку y_k внутри допустимого множества, которая минимизирует эту линейную модель. Эта подзадача называется **линейным оракулом** (Linear Minimization Oracle, LMO):

$$y_k = \arg \min_{\|x\|_1 \leq R} \langle \nabla f(x_k), x \rangle \quad (7)$$

Поскольку мы минимизируем линейную функцию на выпуклом многограннике, минимум всегда достигается в одной из его вершин.

После того как целевая вершина y_k найдена, метод обновляет текущую точку, беря выпуклую комбинацию старой точки x_k и найденной вершины y_k :

$$x_{k+1} = \gamma_k x_k + (1 - \gamma_k) y_k \quad (8)$$

где $\gamma_k \in [0, 1)$ задает вес старой точки. Классическим выбором для метода Франк-Вульфа является шаг:

$$\gamma_k = \frac{k-1}{k+1}, \quad k \geq 1 \quad (9)$$

Так как x_{k+1} является выпуклой комбинацией двух точек из L_1 -шара, новая точка гарантированно останется внутри множества.

2.3 Методы внутренней точки

2.3.1 Концепция метода барьеров

Основная идея метода барьеров заключается в сведении исходной условной задачи (6) к последовательности специальным образом построенных безусловных задач, решения которых сходятся к решению исходной условной задачи. Это делается с помощью барьерной функции.

Пусть Q — множество, задаваемое функциональными ограничениями $g_1, \dots, g_m$:

$$Q := \{x \in \mathbb{R}^n : g_i(x) \leq 0 \text{ для всех } 1 \leq i \leq m\}.$$

Барьерной функцией (или функцией внутренних штрафов) для множества Q называется любая функция $F : \text{int } Q \rightarrow \mathbb{R}$, определенная на внутренности множества Q и являющаяся достаточно гладкой, которая обладает барьерным свойством: $F(x) \rightarrow +\infty$ при приближении x к границе множества Q изнутри.

Имея в распоряжении барьерную функцию, определим для каждого $t \geq 0$ вспомогательную функцию $f_t : \text{int } Q \rightarrow \mathbb{R}$ по формуле

$$f_t(x) := tf(x) + F(x)$$

Оказывается, что при $t \rightarrow +\infty$ точка $x^*(t)$ сходится к множеству решений исходной условной задачи (6). Обозначим $x^*(t) := \operatorname{argmin}_{x \in \text{int } Q} f_t(x)$. Множество точек $\{x^*(t) : t \geq 0\}$ называется центральным путем. Таким образом, для решения исходной условной задачи (6), можно использовать следующую схему отслеживания пути (она же и есть метод барьеров).

Общая схема метода барьеров

- (a) Выбрать начальное приближение $x_0 \in \text{int } Q$ и начальный параметр $t_0 > 0$.
- (b) На каждой итерации $k \geq 0$:

- 1. Вычислить точку

$$x_{k+1} := \operatorname{argmin}_{y \in \text{int } Q} f_{t_k}(y), \quad (10)$$

используя некоторый метод безусловной оптимизации с начальной точкой x_k .

- 2. Увеличить параметр t : выбрать $t_{k+1} > t_k$ (чтобы в итоге $t_k \rightarrow +\infty$).

2.3.2 Решение вспомогательной задачи

Заметим, что задача оптимизации, возникающая на каждой итерации метода барьеров в реальности является “безусловной”, несмотря на присутствие в этой задаче условия $x \in \text{int } Q$. Действительно, за счет барьерного свойства целевая функция f_t стремится к $+\infty$ при приближении к границе множества Q , поэтому никакой “разумный” метод оптимизации достаточно близко к границе множества Q никогда не подойдет, и, с точки зрения самого метода, целевая функция как бы задана всюду. Тем не менее, здесь необходима определенная аккуратность.

Например, пусть для решения вспомогательной задачи (10) используется метод спуска: сначала $y_0 := x_k$, далее на каждой итерации $\ell \geq 0$ строится направление спуска $d_\ell \in \mathbb{R}^n$ для функции f_{t_k} в точке y_ℓ , и выполняется шаг

$$y_{\ell+1} := y_\ell + \alpha_\ell d_\ell,$$

где $\alpha_\ell \geq 0$ — длина шага. Поскольку метод барьеров по построению всегда работает с внутренними точками множества Q (отсюда и альтернативное название - метод внутренней точки), то для всех достаточно маленьких длин шагов α_ℓ новая точка $y_{\ell+1}$ будет принадлежать множеству Q , а значение функции f_{t_k} в этой точке уменьшится по сравнению с текущей точкой y_ℓ . Однако, если длина шага α_ℓ выбрана слишком большой, то точка $y_{\ell+1}$ может вообще оказаться за пределами множества Q — такое вполне может произойти при подборе шага в стандартном алгоритме линейного поиска. Чтобы избежать этой проблемы, стандартную процедуру линейного поиска необходимо модифицировать запретив ей вычислять функцию в точках за пределами множества Q .

Согласно определению множества Q , чтобы $y_\ell + \alpha d_\ell \in \text{int } Q$, длина шага $\alpha \geq 0$ должна удовлетворять следующей системе неравенств:

$$g_i(y_\ell + \alpha d_\ell) < 0, \quad 1 \leq i \leq m.$$

Наиболее просто эта система решается в том случае, когда множество Q является полиэдром, т. е. когда функции $g_1, \dots, g_m$ являются аффинными: $g_i(x) = \langle q_i, x \rangle - s_i$. В этом случае соответствующая система неравенств выглядит следующим образом:

$$\langle q_i, y_\ell + \alpha d_\ell \rangle < s_i, \quad 1 \leq i \leq m.$$

Раскрывая скобки, получаем

$$\langle q_i, d_\ell \rangle \alpha < s_i - \langle q_i, y_\ell \rangle, \quad 1 \leq i \leq m \quad (11)$$

Поскольку по построению точка y_ℓ — внутренняя, то все правые части в этих неравенствах строго положительные. Значит, если для некоторого $1 \leq i \leq m$ коэффициент $\langle q_i, d_\ell \rangle \leq 0$, то соответствующее неравенство бессодержательно (поскольку $\alpha \geq 0$). Введем обозначение

$$I_\ell := \{1 \leq i \leq m : \langle q_i, d_\ell \rangle > 0\}.$$

Тогда система (11) эквивалентна следующей системе:

$$\langle q_i, d_\ell \rangle \alpha < s_i - \langle q_i, y_\ell \rangle, \quad i \in I_\ell.$$

Заметим, что теперь все коэффициенты при α строго положительные, поэтому законно разделить:

$$\alpha < \frac{s_i - \langle q_i, y_\ell \rangle}{\langle q_i, d_\ell \rangle}, \quad i \in I_\ell.$$

Наконец, последнюю систему можно переписать в виде одного скалярного неравенства:

$$\alpha < \alpha_\ell^{\max}, \quad \text{где } \alpha_\ell^{\max} := \min_{i \in I_\ell} \frac{s_i - \langle q_i, y_\ell \rangle}{\langle q_i, d_\ell \rangle}.$$

Итак, согласно проведенным выкладкам, все допустимые длины шагов α лежат в интервале $[0, \alpha_\ell^{\max})$, т. е. в процедуре линейного поиска никогда нельзя брать шаг $\alpha \geq \alpha_\ell^{\max}$. Например, если для подбора длины шага используется процедура бэктрекинга, то в этом случае единственная модификация, которую нужно сделать по сравнению со стандартным случаем, — это изменить начальную пробную длину шага: вместо прежде используемого значения α_ℓ^0 теперь нужно использовать значение

$$\tilde{\alpha}_\ell^0 := \min \{ \alpha_\ell^0, \theta \alpha_\ell^{\max} \},$$

где $0 < \theta < 1$ — некоторая константа (например, 0.99).

2.3.3 Полная спецификация метода

Несмотря на то, что гипотетически в методе барьеров можно использовать любую комбинацию барьерной функции и метода безусловной минимизации, наиболее эффективной (как с точки зрения теории, так и практики) оказывается связка логарифмического барьера и метода Ньютона. Логарифмический барьер в общем виде имеет вид

$$F(x) := - \sum_{i=1}^m \ln(-g_i(x))$$

В качестве критерия останова в методе Ньютона в этом задании Вам нужно будет использовать стандартный критерий по норме градиента целевой функции (который использовался в предыдущих лабораторных работах) :

$$\|\nabla f_{t_k}(y_\ell)\|^2 \leq \epsilon_{\text{inner}} \|\nabla f_{t_k}(x_k)\|^2$$

где $\epsilon_{\text{inner}} > 0$ - некоторый параметр, который обычно выбирается довольно маленьким (например, 10^{-7}), что соответствует практически точному решению внутренней задачи (в выпуклом случае). Заметим, что поскольку в качестве алгоритма минимизации используется метод Ньютона, а стартовая точка выбирается достаточно «хорошей», то маленькое значение ϵ_{inner} совсем не представляет никакой проблемы для метода (напомним, что метод Ньютона имеет локальную квадратичную сходимость).

Что касается стратегии увеличения параметров t_k , то здесь имеется несколько эффективных способов. Самый простой из них, которую Вам нужно будет использовать в этом задании, состоит в постоянном линейном увеличении:

$$t_{k+1} := \gamma t_k$$

где $\gamma > 1$ — постоянная константа (обычно порядка 10-100).

Функция $\|x\|_1 = \sum_{i=1}^n |x_i|$ является негладкой в нуле. Мы можем ввести вектор дополнительных (вспомогательных) переменных $u \in \mathbb{R}^n$, который будет играть роль верхней оценки для модулей компонент вектора x . Заметим, что условие $|x_i| \leq u_i$ эквивалентно системе из двух линейных неравенств: $x_i \leq u_i$ и $-x_i \leq u_i$.

Тогда исходная задача переписывается в эквивалентном виде:

$$\min_{x, u \in \mathbb{R}^n} \left[f(x) + \lambda \sum_{i=1}^n u_i \right] \quad \text{при условии} \quad x_i - u_i \leq 0, \quad -x_i - u_i \leq 0, \quad \forall i = 1 \dots n \quad (12)$$

В этой постановке целевая функция линейна по u и наследует гладкость от $f(x)$ по x . Все ограничения являются аффинными. Пространство оптимизируемых параметров теперь имеет размерность $2n$, а вектор неизвестных есть $z = [x, u]^T$.

Для задачи (12) логарифмический барьер будет иметь следующий вид:

$$F(x, u) = - \sum_{i=1}^n \ln(u_i - x_i) - \sum_{i=1}^n \ln(u_i + x_i) \quad (13)$$

Область определения этой функции $\text{dom} F = \{(x, u) \mid u_i > |x_i|\}$. Пока мы находимся внутри этой области (в строгой внутренней), функция F конечна и бесконечно дифференцируема.

Формируем новую барьерную целевую функцию $f_t(x, u)$:

$$f_t(x, u) = t \left(f(x) + \lambda \sum_{i=1}^n u_i \right) + F(x, u) \longrightarrow \min_{x, u} \quad (14)$$

По мере увеличения параметра $t \rightarrow \infty$, минимум функции $f_t(x, u)$ сходится к истинному решению условной задачи (12).

Метод барьеров дает нам следующее свойство: в оптимуме вспомогательной задачи для параметра t , зазор между текущим значением $f(x) + \lambda \|x\|_1$ и истинным глобальным минимумом строго равен $\frac{2n}{t}$, где $2n$ – общее число ограничений-неравенств. Это дает аналитический критерий останова всего алгоритма.

3 Формулировка задания

Как и в предыдущих лабораторных работах, задание выполняется в командах по 3 человека. Каждая команда проводит базовые эксперименты на ML-оракулах из своего варианта (закрепленного с Лабораторной №1) и выполняет индивидуальный исследовательский трек по вариантам из Приложения А. В отчете необходимо явно указать, как были распределены задачи между участниками команды.

3.1 План выполнения

1. Возьмите реализованные вами ранее классы гладких ML-оракулов. На их основе в модуле `oracles`:
 - Реализуйте композитные оракулы для L_1 регуляризаторами. - Реализуйте функцию проксимального оператора для L_1 -регуляризатора
 - Реализуйте линейный оракул (ЛМО) для L_1 -шара
 - Реализуйте вычисление градиента и гессиана барьерной функции $F_t(x, u)$ размера $2n \times 2n$.
2. Реализуйте Субградиентный метод (с убывающим шагом $\alpha_k = \frac{\alpha_0}{\sqrt{k+1}}$) и методы ISTA/FISTA (с бэктрекингом по L): (функции `subgradient_method`, `proximal_gradient_method`, `proximal_fast_gradient_method` в модуле `optimization`)
3. Реализуйте метод Франк-Вульфа. Рассмотрите две стратегии выбора длины шага γ_k : классическую убывающую последовательность $\frac{k-1}{k+1}$ и подбор шага с помощью линейного поиска (условие Армихо) (функция `frank_wolfe_method` в модуле `optimization`).
4. Реализуйте метод логарифмических барьеров. В качестве внутреннего решателя используйте Метод Ньютона. Обязательно внедрите осторожный бэктрекинг: шаг α должен быть строго внутри области определения логарифмов ($u_i > |x_i|$) (функция `barrier_method` в модуле `optimization`).
5. Поскольку метод Франк-Вульфа решает задачу с ограничением $\|x\|_1 \leq R$, а остальные методы – задачу со штрафом $\lambda \|x\|_1$, для корректного сравнения необходимо найти соответствие параметров. Выберите некоторое $\lambda > 0$, запустите FISTA до полной сходимости и получите оптимальное решение x_λ^* . Затем установите радиус для метода Франк-Вульфа $R = \|x_\lambda^*\|_1$.
6. Проведите эксперименты, описанные в разделах 3.2 – 3.4. Оформите результаты в отчет.
7. Выполните один индивидуальный исследовательский трек на всю команду (см. Приложение А).

Критерии останова: Так как задача негладкая, норма градиента неприменима. Во всех базовых экспериментах используйте следующие специфичные для методов критерии:

- *Субградиентный и FISTA*: Относительное изменение весов $\frac{\|x_{k+1}-x_k\|_2}{\max(1,\|x_k\|_2)} \leq \epsilon$.
- *Франк-Вульф*: Зазор Франк-Вульфа $\langle \nabla f(x_k), x_k - y_k \rangle \leq \epsilon$ (где y_k – найденная вершина).
- *Метод барьеров*: Барьерный зазор $\frac{2n}{t} \leq \epsilon$ (для внешнего цикла). Внутренний метод Ньютона останавливать по относительной норме градиента.

3.2 Эксперимент 1

Цель эксперимента — проверить, способны ли методы выдавать строгую разреженность векторов.

1. Подберите параметр λ в ваших ML-оракулах и тестовых датасетах так, чтобы в оптимальном решении x^* примерно 50 – 80% весов были строго нулевыми.
2. Запустите все 4 метода (Субградиентный, FISTA, Франк-Вульф, Метод барьеров) из начальной точки $x_0 = 0$.
3. Постройте график: по оси X — номер итерации (или время), по оси Y -доля строго нулевых компонент (в %) в векторе x_k . *Примечание: для методов, которые не дают точных нулей, считайте нулем значения $|x_i| < 10^{-8}$.*

Какой метод (или методы) по своей математической природе способен сразу выдавать точные нули 0.0, не прибегая к эвристическому отсечению мелких чисел? Почему субградиентный метод не дает нулей? Почему метод барьеров в процессе оптимизации всегда держит веса ненулевыми?

3.3 Эксперимент 2

1. Возьмите обе ваши ML-задачи (регрессию и классификацию). Используйте синхронизированные параметры λ и R (см. п. 3.1).
2. Чтобы получить оптимум F^* , запустите FISTA на 10 000 итераций.
3. Запустите все 4 метода и сохраняйте значение целевой функции $F(x_k) = f(x_k) + \lambda \|x_k\|_1$.
4. Постройте графики сходимости двух видов:
 - (а) Зависимость логарифма ошибки $\log(F(x_k) - F^*)$ от числа вызовов оракула (для метода барьеров считайте вызовы внутри Ньютона).
 - (б) Зависимость логарифма ошибки $\log(F(x_k) - F^*)$ от реального времени работы в секундах.

(Все методы должны быть отображены на одном графике разными цветами).

Какой подход оказался самым быстрым по числу итераций, а какой - по времени в секундах? Как тяжесть одной итерации (поиск вершины, проксимальный оператор, обращение гессиана) сказывается на итоговом времени?

3.4 Эксперимент 3

В этом эксперименте нужно протестировать «чувствительность» методов на одном из датасетов обеих задач (регрессии и классификации). Постройте три отдельных графика сходимости $\log(F(x_k) - F^*)$ от числа итераций:

1. Наложите на один график траекторию классического проксимального градиентного спуска (ISTA) и ускоренного (FISTA). Насколько сильно инерция ускоряет сходимость? Наблюдаете ли вы осцилляции на графике FISTA?
2. Наложите на один график метод Франк-Вульфа с классическим шагом $\gamma_k = \frac{k-1}{k+1}$ и метод Франк-Вульфа с линейным поиском по условию Армихо. Окупают ли себя дополнительные вызовы оракула при линейном поиске?
3. Запустите метод барьеров с разными коэффициентами увеличения штрафа: $t \in \{2, 10, 50, 100\}$. Постройте график зависимости суммарного числа внутренних ньютоновских итераций от выбранного t . Существует ли оптимальный баланс между частым обновлением барьера и тяжестью внутренних подзадач?

3.5 Углубленное исследование: Индивидуальный трек команды

В данном разделе команда должна представить результаты исследования по своему индивидуальному треку (см. Приложение А). Это задание требует постановки отдельного вычислительного эксперимента и глубокого математического анализа.

Отчет по треку должен содержать:

1. **Математическую постановку:** Вывод формул, доказательства или теоретическое описание исследуемой проблемы.
2. **Описание эксперимента:** Какие параметры варьировались, на каких данных проводились тесты.
3. **Результаты:** Графики, таблицы, профили времени работы.
4. **Выводы:** Глубокая аналитика полученных результатов. Почему метод вел себя именно так? Подтверждают ли результаты геометрию оптимизации из лекций?

4 Оформление задания

Работа команды сдается в виде трех взаимосвязанных артефактов (см. Лабораторная работа 1):

- Репозиторий на Github/Gitlab
- Отчет в PDF
- Презентация

Каждый проведенный эксперимент следует оформить как отдельный раздел в PDF документе (название раздела - название соответствующего эксперимента). Для каждого эксперимента необходимо сначала написать его описание:

- а) постановка задачи
- б) какие функции оптимизируются, каким образом генерируются данные, Описание датасетов (размерность $m \times n$, плотность/разреженность матриц), используемого "железа"(CPU/RAM), какие методы и с какими параметрами используются.
- в) далее должны быть представлены результаты соответствующего эксперимента - графики, таблицы и т. д.

г) наконец, после результатов эксперимента должны быть написаны Ваши выводы - какая зависимость наблюдается и почему. Как это бьется с теорией оптимизации из лекций?

Важно: Отчет не должен содержать никакого кода. Каждый график должен быть прокомментирован - что на нем изображено, какие выводы можно сделать из этого эксперимента. Обязательно должны быть подписаны оси. Если на графике нарисовано несколько кривых, то должна быть легенда. Сами линии следует рисовать достаточно толстыми, чтобы они были хорошо видимыми.

Приложение А. Индивидуальные исследовательские треки

Трек 1. Проксимальный метод Ньютона ²

Описание

Методы первого порядка, такие как FISTA, гарантируют оптимальную для своего класса скорость сходимости $\mathcal{O}(1/k^2)$. Однако на практике их производительность сильно деградирует при плохой обусловленности гладкой части $f(x)$ (когда признаки сильно скоррелированы, а линии уровня вытянуты). В безусловной оптимизации эту проблему решает Метод Ньютона.

Однако мы не можем напрямую применить классический шаг Ньютона к нашей задаче из-за негладкости регуляризатора $\|x\|_1$. Идея Проксимального метода Ньютона заключается в построении на каждой итерации k квадратичной аппроксимации Тейлора *только для гладкой части* $f(x)$, оставляя регуляризатор без изменений. Новое направление шага d_k находится как решение следующей вспомогательной негладкой задачи:

$$d_k = \arg \min_{d \in \mathbb{R}^n} \left[\langle \nabla f(x_k), d \rangle + \frac{1}{2} d^T \nabla^2 f(x_k) d + \lambda \|x_k + d\|_1 \right] \quad (15)$$

Эта подзадача сама по себе является задачей типа Lasso (квадратичная форма плюс L_1 -штраф). Хотя у неё нет простого аналитического решения в виде одного вызова проксимального оператора, её можно очень эффективно решить итерационно, так как гессиан $\nabla^2 f(x_k)$ вычисляется лишь один раз и остается константой внутри подзадачи. После нахождения направления d_k выполняется шаг $x_{k+1} = x_k + \alpha d_k$, где α подбирается линейным поиском.

Задание

1. Выведите формулу квадратичной подзадачи (15) для ваших ML-моделей.
2. Реализуйте Проксимальный метод Ньютона. Для решения внутренней подзадачи используйте реализованный вами ранее метод FISTA.
3. Поскольку подзадача решается итерационно, для общей эффективности алгоритма не нужно решать её с высокой точностью на первых внешних итерациях. Внедрите стратегию *адаптивной точности*: останавливайте внутренний метод, когда норма невязки подзадачи станет меньше, чем η_k , где η_k убывает по мере приближения x_k к глобальному оптимуму.

²https://papers.nips.cc/paper_files/paper/2012/file/0f304eddb4ad6007a3093fd6d963a1d2-Paper.pdf

4. Сравните полученный метод с базовым FISTA на датасете с плохой обусловленностью (сильно скоррелированными признаками).

Вопросы для анализа в отчете

1. Почему метод Ньютона в композитной постановке требует запуска внутреннего итерационного цикла, в то время как в гладкой задаче (лабораторная работа №1) он требовал лишь одного решения СЛАУ?
2. Как обусловленность матрицы Гессе $\nabla^2 f(x)$ влияет на количество итераций *внутреннего* решателя?
3. Оправдывает ли себя использование второго порядка (дорогое вычисление гессиана и запуск внутренних циклов), если сравнить итоговое время работы (в секундах) Проксимального Ньютона и FISTA? На каких данных Ньютон выигрывает?

Трек 2. Метод Франк-Вульфа с отшагиванием (Away-steps FW)

Описание

Классический метод Франк-Вульфа (FW) имеет сублинейную скорость сходимости $\mathcal{O}(1/k)$. Это математическое ограничение связано с тем, что алгоритм может только добавлять новые вершины в выпуклую комбинацию текущей точки $x_k = \sum \alpha_i v_i$. Если оптимальное решение лежит на грани L_1 -многогранника (что означает разреженность), метод начинает «зигзагообразно» прыгать между вершинами, образующими эту грань, не в силах быстро обнулить коэффициенты α_i у тех вершин, которые в процессе оптимизации стали «плохими».

Модификация **Away-steps Frank-Wolfe** AFW³ элегантно решает эту проблему геометрии. На каждом шаге алгоритм выбирает между двумя направлениями:

- **FW-step:** Движение в сторону наилучшей вершины y_{FW} (как в обычном методе).
- **Away-step:** Движение прочь от наихудшей вершины v_{away} , которая в данный момент входит в активное множество текущей точки x_k с положительным весом $\alpha_i > 0$.

Это позволяет методу активно «выбрасывать» ошибочные вершины из разложения. Для L_1 -шара радиуса R вершинами являются базисные орты $\pm Re_i$.

Задание

1. Реализуйте алгоритм AFW. Для этого вам необходимо поддерживать активное множество вершин и их барицентрические координаты (веса) α_i , такие что $x_k = \sum \alpha_i v_i$ и $\sum \alpha_i = 1$.
2. Реализуйте логику выбора направления на каждой итерации: алгоритм вычисляет скалярное произведение градиента как с наилучшей возможной вершиной (для FW-step), так и с наихудшей вершиной из активного множества (для Away-step). Выполняется тот шаг, который обещает большее убывание линейной модели.

³<http://papers.neurips.cc/paper/5925-on-the-global-linear-convergence-of-frank-wolfe-optimization.pdf>

3. **Важно:** При выполнении Away-step нельзя допустить, чтобы вес вершины α_{away} стал отрицательным. Реализуйте так называемый «drop step» – если шаг слишком большой, вес α_{away} просто обнуляется, а сама вершина навсегда удаляется из активного множества.
4. Запустите классический FW и AFW на задаче с большим параметром разреженности (где большинство весов в оптимуме нули).

Вопросы для анализа в отчете

1. Как меняется размер активного множества (количество вершин с ненулевым весом) в процессе работы AFW по сравнению с обычным FW?
2. Постройте график логарифма невязки от номера итерации. Продемонстрируйте переход от сублинейной сходимости (убывание как $\sim 1/k$) к линейной (убывание прямой линией в лог-масштабе) при использовании Away-steps.
3. Почему для эффективной реализации AFW критически важно хранить само активное множество, а не просто поддерживать текущий вектор x_k в памяти?

Трек 3. Покоординатный спуск (Coordinate Descent)

Описание

Покоординатный спуск (CD) является стандартом для решения задач с L_1 -регуляризацией (именно он реализован «под капотом» библиотек `scikit-learn` и `glmnet` для Lasso и ElasticNet).

Вместо того чтобы делать шаг сразу по всем измерениям вектора $x \in \mathbb{R}^n$, метод CD на каждой итерации фиксирует все переменные, кроме одной x_i , и минимизирует целевую функцию *строго по этой одной координате*:

$$x_i^{new} = \arg \min_{z \in \mathbb{R}} [f(x_1, \dots, x_{i-1}, z, x_{i+1}, \dots, x_n) + \lambda |z|] \quad (16)$$

Прелесть этого подхода для L_1 -регуляризации состоит в том, что 1D-подзадача часто имеет аналитическое решение через оператор Soft-Thresholding. Даже если функция f сложная (например, логистическая регрессия), её можно аппроксимировать параболой по i -й координате, сведя шаг к элементарной алгебраической операции. Обновляя координаты одну за другой, метод совершает зигзагообразное, но математически точное движение вдоль осей координат, что идеально совпадает с геометрией L_1 -шара.

Задание

1. Выведите аналитическое правило обновления x_i для вашей ML-функции. Если точного минимума по координате нет, выведите шаг обновления, используя минимизацию квадратичной мажоранты (верхней оценки) по i -й координате.
2. Реализуйте алгоритм Cyclic Coordinate Descent (обновление координат строго по кругу: $1, 2, \dots, n, 1, 2, \dots$).
3. Оптимизируйте вычисления: при изменении одной координаты x_i , полное пере вычисление скалярных произведений $\langle a_j, x \rangle$ для всей выборки занимает $\mathcal{O}(m \cdot n)$. Реализуйте поддержку «инварианта» (вектора предсказаний), обновление которого при изменении одного x_i будет стоить всего $\mathcal{O}(m)$.

4. Сравните эффективность CD и FISTA.

Вопросы для анализа в отчете

1. Покажите математический вывод шага покоординатного спуска для вашей функции. Где именно в формуле возникает оператор мягкого порога, гарантирующий обнуление координаты?
2. Постройте график сходимости методов от реального времени работы (секунды). Почему алгоритм, обновляющий лишь одну координату за раз, способен обгонять градиентные методы, вычисляющие полный вектор направления?
3. Как степень разреженности матрицы признаков A (sparse matrix) влияет на вычислительную сложность одной эпохи покоординатного спуска по сравнению с вычислением полного градиента?

Трек 4. Обусловленность барьера и L-BFGS

Описание

В безусловной оптимизации квазиньютоновские методы (такие как L-BFGS) часто являются алгоритмами по умолчанию, так как они строят аппроксимацию гессиана исключительно на основе истории градиентов, экономя огромное количество вычислений. Но не для метода логарифмических барьеров ⁴.

По мере приближения алгоритма к границе допустимого множества (когда $u_i \approx |x_i|$) и роста барьерного параметра $t \rightarrow \infty$, гессиан барьерной функции $\nabla^2 F_t$ становится катастрофически плохо обусловленным: его собственные значения разрываются на порядки. Классический Метод Ньютона обладает свойством аффинной инвариантности и справляется с плохой обусловленностью за счет прямого решения системы уравнений. L-BFGS же работает в евклидовой метрике, и искажение пространства барьером полностью ломает его способность находить правильное направление убывания.

Задание

1. Встройте в ваш реализованный Метод барьеров возможность переключения внутреннего решателя. Используйте ваш метод L-BFGS из Лабораторной №2.
2. Запустите Метод барьеров дважды: с внутренним Методом Ньютона и с внутренним L-BFGS.
3. На каждой внешней итерации t вычисляйте число обусловленности точного гессиана барьерной функции: $\kappa = \frac{\lambda_{\max}(\nabla^2 F_t)}{\lambda_{\min}(\nabla^2 F_t)}$.
4. Отследите, сколько итераций требует каждый из внутренних методов для достижения сходимости по мере роста t .

⁴Nocedal J., Wright S. *Numerical Optimization*. (Главы 6 и 19).

Вопросы для анализа в отчете

1. Постройте график зависимости числа обусловленности κ от параметра t (в логарифмическом масштабе). Как теоретически и практически растет κ при приближении к границе?
2. В какой момент (при каком t) внутренний метод L-BFGS начинает требовать аномально много итераций или вовсе ломается (не может найти шаг по условию Вульфа)?
3. Объясните математически, опираясь на свойства аффинной инвариантности, почему Метод Ньютона практически не замечает ухудшения обусловленности матрицы Гессе барьерной функции?

Трек 5. Метод переменных направлений (ADMM)

Описание

Альтернативный и очень мощный способ справиться с композитной задачей заключается в применении техники расщепления переменных. Мы можем ввести искусственную переменную $z \in \mathbb{R}^n$ и переписать задачу так, чтобы разделить гладкую и негладкую части:

$$\min_{x, z \in \mathbb{R}^n} f(x) + \lambda \|z\|_1 \quad \text{при условии} \quad x - z = 0 \quad (17)$$

Метод **ADMM (Alternating Direction Method of Multipliers)**⁵ решает эту задачу путем построения аугментированного лагранжиана: $\mathcal{L}_\rho(x, z, y) = f(x) + \lambda \|z\|_1 + y^T(x - z) + \frac{\rho}{2} \|x - z\|_2^2$.

Алгоритм минимизирует лагранжиан поочередно по переменным x и z , а затем обновляет двойственную переменную y . Красота подхода в том, что шаг по x сводится к минимизации гладкой функции (где можно применить L-BFGS или Ньютона), а шаг по z имеет аналитическое решение через проксимальный оператор.

Задание

1. Выпишите шаги ADMM для вашей задачи. Аналитически докажите, что минимизация $\mathcal{L}_\rho$ по переменной z строго сводится к оператору Soft-Thresholding.
2. Реализуйте алгоритм ADMM. Для решения гладкой подзадачи по x используйте любой градиентный метод, L-BFGS или Ньютона.
3. Оптимизируйте метод: подзадача по x не требует точного решения на каждой итерации.
4. Запустите ADMM и исследуйте его чувствительность к гиперпараметру штрафа $\rho > 0$.

⁵https://web.stanford.edu/~boyd/papers/pdf/admm_distr_stats.pdf

Вопросы для анализа в отчете

1. Покажите математический вывод шага по z .
2. Постройте на одном графике сходимость primal residual ($\|x_k - z_k\|_2$) и dual residual (изменение z_k). Как параметр ρ смещает баланс между скоростью выполнения прямого и двойственного условий?
3. Сравните время работы ADMM и FISTA до достижения одинаковой точности. Как вы думаете почему ADMM особенно популярен в распределенных системах оптимизации?

Трек 6. Адаптивные рестарты в методах Нестерова

Описание

Ускоренный метод Нестерова (включая композитный FISTA) достигает скорости $\mathcal{O}(1/k^2)$ для выпуклых задач за счет накопления «инерции» (момента). Однако для сильно выпуклых или кусочно-линейных функций эта инерция может сыграть злую шутку: метод «пролетает» мимо оптимума, после чего вынужден разворачиваться. Это приводит к характерным осцилляциям (ряби) на графике сходимости и падению скорости на поздних этапах.

Эту проблему решает механизм адаптивных рестартов⁶. Идея заключается в постоянном мониторинге поведения метода: если алгоритм начинает двигаться в «неправильном» направлении (угол между направлением движения и антиградиентом становится тупым), мы принудительно обнуляем накопленную инерцию ($v_k \leftarrow x_k, t_k \leftarrow 1$) и начинаем ускорение заново.

Задание

1. Реализуйте FISTA с эвристикой градиентного рестарта. Сброс инерции происходит, если:

$$\langle y_k - x_{k+1}, x_{k+1} - x_k \rangle > 0 \quad (18)$$

(Здесь $y_k - x_{k+1}$ является аналогом градиентного шага, а $x_{k+1} - x_k$ — фактическое направление движения метода).

2. Реализуйте альтернативную эвристику рестарта по функции: сброс происходит, если целевая функция строго возросла: $F(x_{k+1}) > F(x_k)$.
3. Сравните базовый FISTA и FISTA с рестартами на датасете, где базовая версия показывает сильные колебания.

Вопросы для анализа в отчете

1. Постройте график логарифма невязки по целевой функции. Ярко продемонстрируйте «горбы» (осцилляции) на графике обычного FISTA и то, как рестарты "срезают" эти горбы.
2. Объясните геометрический смысл градиентного условия рестарта. Почему положительное скалярное произведение сигнализирует о том, что инерция тянет нас не туда?

⁶<https://arxiv.org/abs/1204.3982>

3. Какой из двух критериев рестарта (градиентный или по функции) оказался более стабильным и эффективным на ваших ML-задачах?

Трек 7. Сглаживание Нестерова и Проксимальные методы

Описание

За несколько лет до создания FISTA Ю.Е. Нестеров предложил принципиально иной подход к негладкой оптимизации: вместо использования проксимальных операторов, можно аналитически сгладить негладкую функцию $h(x)$, заменив её дифференцируемой аппроксимацией $h_\mu(x)$ (сглаживание Моро-Иосиды)⁷.

Для L_1 -регуляризатора $\lambda\|x\|_1$ такой сглаженной аппроксимацией является функция Хьюбера (Huber penalty) с малым параметром сглаживания $\mu > 0$:

$$h_\mu(x_i) = \begin{cases} \frac{x_i^2}{2\mu}, & |x_i| \leq \mu \\ |x_i| - \frac{\mu}{2}, & |x_i| > \mu \end{cases} \quad (19)$$

После сглаживания задача становится полностью дифференцируемой, и к ней можно применить Ускоренный градиентный метод Нестерова (NAG) без проксимальных операторов. Однако возникает фундаментальный компромисс: чем меньше μ , тем точнее мы решаем исходную задачу, но тем хуже константа Липшица сглаженной функции $L_{h_\mu} = \frac{\lambda}{\mu}$. Это заставляет градиентный метод делать микроскопические шаги.

Задание

1. Выведите константу Липшица градиента для сглаженной задачи $F_\mu(x) = f(x) + \sum h_\mu(x_i)$. *Внимание:* Если вашей команде достались оракулы **Poisson Regression** или **Exponential Loss**, глобальной константы Липшица для гладкой части L_f не существует (гессиан не ограничен). В этом случае докажите этот факт в отчете, а в алгоритме NAG обязательно используйте адаптивный линейный поиск (бэктрекинг)!
2. Запустите Ускоренный градиентный метод (NAG) на сглаженной функции для нескольких значений μ (например, $10^{-1}, 10^{-2}, 10^{-4}$).
3. Сравните точность решения и скорость сходимости сглаженного подхода со строгим проксимальным методом FISTA.

Вопросы для анализа в отчете

1. Постройте график зависимости итогового значения негладкой функции $F(x_{end})$ от параметра μ . Как μ влияет на скорость сходимости метода?
2. Удалось ли сглаженному методу получить *абсолютные* нули в весах модели (истинную разреженность)? Объясните, почему функция Хьюбера математически не способна притягивать веса строго в 0.0, в отличие от Soft-Thresholding.
3. Сделайте глобальный вывод: что вычислительно эффективнее и надежнее в машинном обучении – сглаживать функцию и использовать обычные методы или применять негладкие алгоритмы с проксимальным оператором?

⁷Nesterov Y. *Smooth minimization of non-smooth functions*. (Mathematical programming, 2005)